Distributed Acceleration of Hierarchical PCA-QD Genetic-Programming Factor Mining

DSA5208 Project 3

[TODO: Name (Student ID)] [TODO: Name (Student ID)]
[TODO: Name (Student ID)]

14 November 2026

Abstract

We accelerate an existing genetic-programming (GP) factor miner for Chinese A-share stocks. The search loop stays in Python; its batch evaluation engine is re-implemented in C++ and parallelised with MPI + OpenMP (factor computation and evaluation) and with CUDA (PCA signatures and similarity). The data are decomposed by trading date, which turns per-expression parallelism into a genuinely distributed computation: rolling operators need halo exchange or redundant computation, the exponential moving average needs a scan across ranks, and daily statistics need a global reduction. Shared subexpressions of a batch are merged into a DAG, so each is computed once. On the fixed factor-evaluation batch, pure MPI at 288 cores achieves a speed-up of 67.5 and about 23% efficiency; MPI + OpenMP reaches a speed-up of 182 and about 63% efficiency. For the PCA stage, two GPUs on one node are 1.76 times faster than one GPU.

Terminology

Term	Meaning
factor	a formula mapping raw stock data to a daily score per stock
panel	a $T \times N$ array (trading days $\times$ stocks)
RankIC	daily Spearman correlation between a factor and future returns
fitness	the GP objective computed from the RankIC (Section 1.4)
gene pool	the best expressions of a stage, reused as subtrees by deeper stages
PCA-QD signature	a short vector summarising a factor, used to measure similarity
DAG	directed acyclic graph of the distinct subexpressions of a batch
root	the DAG node of a whole expression (a candidate factor)
halo	history rows a rank needs from the ranks to its left
H_0	redundant-computation budget (days of history recomputed locally)
ema	exponential moving average
strong / weak scaling	fixed total problem / fixed problem per rank
Karp–Flatt metric	experimentally determined serial fraction
GEMV / GEMM	matrix–vector / matrix–matrix multiplication

1 Background and algorithm

1.1 What the miner does

Factors in quantitative stock selection. A quantitative equity strategy ranks thousands of stocks every day and buys the ones it expects to outperform. The ranking is built from *factors*: formulas that turn raw market data (prices, volumes, money flows, intraday statistics) into one

score per stock per day, such as "10-day average of volume divided by amount". A factor is valuable when its daily ranking of stocks agrees with the ranking of their future returns. Practitioners combine many such factors in a multi-factor model, so the goal is not one good formula but a large library of good and mutually different ones.

Why genetic programming. Hand-crafting factors is slow and limited by the researcher's ideas. Every factor is an expression tree over raw fields and a small set of operators, so factor discovery can be posed as a search over trees. The space is combinatorial (Section 1.3: about 10^{12} trees of depth 2 and 10^{20} of depth 3), the objective is computed by evaluating the formula on years of data and is neither smooth nor differentiable, and the result must remain a readable formula. Genetic programming (GP) fits these constraints: it keeps a population of expression trees, selects the fitter ones, and creates new trees by exchanging and mutating subtrees. Its cost is that every generation evaluates thousands of new expressions on the full panel data, which is the workload we parallelise.

Why measure the correlation between factors. GP readily produces many variants of the same idea, e.g. the same signal with a window of 10 or 15 days, or wrapped in a rank. Such near-duplicates score well individually but add almost no information to a multi-factor model, and they crowd out genuinely new ideas in the population. The miner therefore keeps a *diversity archive* (quality-diversity, QD): a new factor is admitted only if it is not too correlated with any archived factor. Correlating full panels pair by pair would cost $O(TN)$ per pair for thousands of candidates against thousands of archived factors, so each factor is compressed by a principal-component (PCA) step into a short signature vector, and similarity is measured between signatures (Section 1.6).

The workload. One GP generation therefore consists of (i) computing a batch of new factor panels from the raw data, (ii) evaluating each against future returns (daily RankIC and fitness), (iii) computing PCA signatures and their similarity to the archive, and (iv) admitting candidates one by one. Steps (i)–(ii) are the factor computation and evaluation part, parallelised with MPI + OpenMP; step (iii) is the PCA part, parallelised on GPUs.

1.2 Factors and RankIC

Every raw field is a panel $X \in \mathbb{R}^{T\times N}$ (days $\times$ stocks). A factor F is computed from raw panels by an expression tree, for example $\mathrm{mean}(\mathrm{volume}/\mathrm{amount}, 10)$. A factor is useful if, on each day, the ranking of stocks by the factor predicts the ranking of their future returns Y. Its quality on day t is

$$\mathrm{IC}_t = \mathrm{corr}\big(\,\mathrm{rank}(F_{t,\cdot}),\ \mathrm{rank}(Y_{t,\cdot})\big), \qquad \overline{\mathrm{IC}} = \frac{1}{|T_{\mathrm{valid}}|}\sum_t \mathrm{IC}_t. \tag{1}$$

1.3 Expressions and operators

Leaves are raw fields. Window lengths are taken from $\{1, 5, 10, 15, 20, 30, 60\}$. There are 20 operators:

- element-wise: $+, -, \times, \div, \log, |\cdot|$;
- cross-sectional: rank, z-score;
- rolling: mean, std, sum, max, min, median, ema, lag, delta, pct_change;
- pairwise rolling: correlation, beta.

Expressions are normalised: commutative operands are ordered, $\mathrm{rank}(\mathrm{rank}(x)) = \mathrm{rank}(x)$, and $x - x$, x/x are rejected. Trees have at most 11 nodes.

The depth of a tree is the number of operators on its longest path (a raw field has depth 0). With 132 fields there are 74 single-input choices (4 operators without a window, 10 rolling operators

× 7 windows) and 18 two-input choices (4 arithmetic operators, 2 pairwise rolling operators × 7 windows). Counting trees with at most 11 nodes, before normalisation removes equivalent forms:

depth	number of distinct trees
1	3.2×10^5
2	1.9×10^{12}
3	5.4×10^{20}

A stage of the GP evaluates about 10^5 expressions (10 generations of 10,000), so exhaustive search is impossible beyond depth 1 and the evaluation of each generation is the cost to reduce.

1.4 Fitness

A candidate must have a valid IC on at least 80% of days and at most 20% missing values. Let $m_1, \ldots, m_4$ be the mean IC on four consecutive time segments. Then

$$\text{fitness} = |\overline{\text{IC}}| - 0.5\ \text{std}(m_1, \ldots, m_4) - 0.0005 \max(\text{nodes} - 3, 0), \tag{2}$$

and the fitness is multiplied by 0.25 if fewer than 75% of the segments share the sign of $\overline{\text{IC}}$.

1.5 Layer-wise generation

Evolution runs in three stages, with maximum tree depth 1, 2 and 3. Each stage evolves a population of 10,000 for 10 generations:

- parents are drawn from a feature-capped pool, by tournament or by ε-lexicase selection over the four IC segments;
- offspring come from same-level crossover and from feature, operator, window and subtree mutation; each family keeps its two best members;
- random immigrants make up 10% of each generation;
- at the end of a stage, the best 2,000 admitted expressions become a *gene pool*; deeper stages build 70% of their subtrees from these gene pools.

1.6 PCA-QD diversity archive

Many candidates are near-duplicates of each other. Comparing full factor panels pairwise costs $O(TN)$ per pair, which is prohibitive for thousands of candidates against thousands of archived factors. Each candidate is therefore compressed into a short signature:

1. take every 5th day and its cross-sectional percentile ranks (missing values set to 0.5), and centre each row, giving $R \in \mathbb{R}^{T_s \times N}$;
2. run four power iterations for the leading right singular vector v;
3. set $s = Rv/\|Rv\|$, with its sign fixed.

A candidate enters the archive if its |correlation| with every archived signature is below 0.9. Otherwise it replaces its nearest neighbour, but only if its RankIC is better by at least 0.002. Admission is sequential.

1.7 Cost of each part

For one expression on T days and N stocks:

- each operator node costs $O(TN)$: element-wise and sliding-window updates are constant work per cell, except the rolling median, which keeps a sorted window and costs $O(TNw)$ with $w \leq 60$;
- the daily RankIC sorts every day's cross-section, $O(TN \log N)$;
- the signature ranks $T_s = T/5$ sampled days, $O(T_s N \log N)$, and runs $k = 4$ power iterations, $O(T_s N k)$;

- the similarity of C candidates against A archived signatures of length T_s costs $O(C\,A\,T_s)$.

A batch of E expressions costs $O(E\,T\,N\log N)$ in total. The run time should therefore grow linearly in T and in the batch size and slightly faster than linearly in N; Section 6.2.1 compares these slopes with measurements.

1.8 The original implementation and its bottlenecks

The Python engine evaluates each expression independently in a pool of 32 processes:

- shared subtrees are recomputed from raw data every time, because its cache stores only final scores;
- every process holds the full data, and parallelism is limited to the number of expressions in a batch;
- the signature is computed in single-threaded NumPy, and only the final similarity matrix multiplication uses the GPU.

A previous run selected 1481 factors, with mean out-of-sample |RankIC| of 0.080 in 2022.

2 Data

2.1 Real data

The raw data are daily panels of 132 numeric fields for Chinese A-shares; part of the fields are aggregated from minute-frequency data to daily frequency. The label is a forward-return panel, masked to tradable, non-ST stocks. The evolution period (2016–2021) has $T = 1461$ trading days and $N = 5876$ stocks. The number of stocks with data grows from 2636 at the start of 2016 to 4845 at the end of 2021. Panels are exported once to raw little-endian float32 files in date-major layout (element $t \cdot N + s$), about 34 MB per field and 4.3 GB in total. The export reuses the original cleaning code, so the C++ engine sees exactly the values the Python miner sees. The 132 fields fall into five groups:

group	fields	examples
daily quotes	10	open, high, low, close, vwap, vol, amount, pct_chg, turnover_rate, total_share
price limits	3	limit_up_price, limit_down_price, limit_status
market identifier	1	mkt
money flow	18	buy/sell amount and volume by order size (small, medium, large, extra large), net flow
minute-derived	100	intraday return moments and quantiles, realised and up-/down volatility, jump variation, morning/afternoon and opening/closing returns, volume concentration, candle shape, VWAP deviation, return autocorrelation

The 100 minute-derived fields are computed from each stock's one-minute bars of the day and aggregated to one value per day.

2.2 Synthetic data

Weak scaling needs a problem that grows with the number of ranks: with 64 days per rank, 288 ranks need 18,432 days, far more than the 1461 real days. Synthetic panels are generated on demand, so a rank produces only its own rows. Every cell depends only on (seed, field, global day, stock), so results do not depend on the number of ranks:

- each field is either log-normal, $\exp(\mu + 0.5z)$ with $\mu \in [-2, 6]$ (like prices, volumes and amounts), or normal, σz with $\sigma \in [0.01, 1]$ (like returns and flows), chosen by a hash of the field name;

- the label is $0.02z$;
- $z \sim N(0,1)$ is drawn independently per cell, and 2% of cells are missing.

Unlike the real data, the number of stocks per day is constant, so synthetic runs have no load imbalance from a growing stock universe.

3 Computing platform: Vanda

All experiments run on the NUS Vanda cluster, which is managed by PBS.

- **Nodes.** A CPU node has two Intel Xeon Platinum 8452Y processors (2×36 cores) and about 500 GiB of memory (501 GiB reported by PBS). GPU nodes have 72 cores and two NVIDIA A40 GPUs with 48 GB each. The CPU pool also contains A40 nodes and some 64-core nodes, so every job records the CPU model of its hosts; all reported runs used 8452Y nodes.
- **Queues.** The free queue (`cpu_parallel`) allows at most 160 cores per job, less than three nodes. Runs with up to four whole nodes (288 cores) therefore use the charged `batch_cpu` queue under a project account, which allows up to 720 cores and 10 nodes per job. Each job requests whole nodes (72 cores, 450 GB) so that no other job shares the memory bandwidth.
- **File systems.** Home and scratch are NFS shares. The data and the results are on scratch, which is purged after 60 days without access, so results are copied back after each run.
- **Software.** GCC 14.3, OpenMPI 5.0.8 and CMake 4.0.3 from environment modules; the engine is built on a compute node. This OpenMPI starts its daemons over ssh rather than through PBS, so a launch agent loads the same modules on every node.
- **Placement.** Ranks are spread evenly over the nodes and each is bound to t consecutive cores (`--map-by ppr:`R/h`:node:PE=`t `--bind-to core`); OpenMP threads are pinned with `OMP_PLACES=cores` and `OMP_PROC_BIND=close`. OpenMPI refuses to bind one rank to cores of both sockets, so for splits where a rank would straddle them ($t > 36$, or 9 ranks of 8 threads per node) the ranks are left unbound and a wrapper pins each rank's threads to its own block of cores. Core numbers alternate between the two sockets (even cores on one, odd on the other), which matters for these splits (Section 4.2.6).

4 Parallel design

4.0 Overview, decomposition and data layout

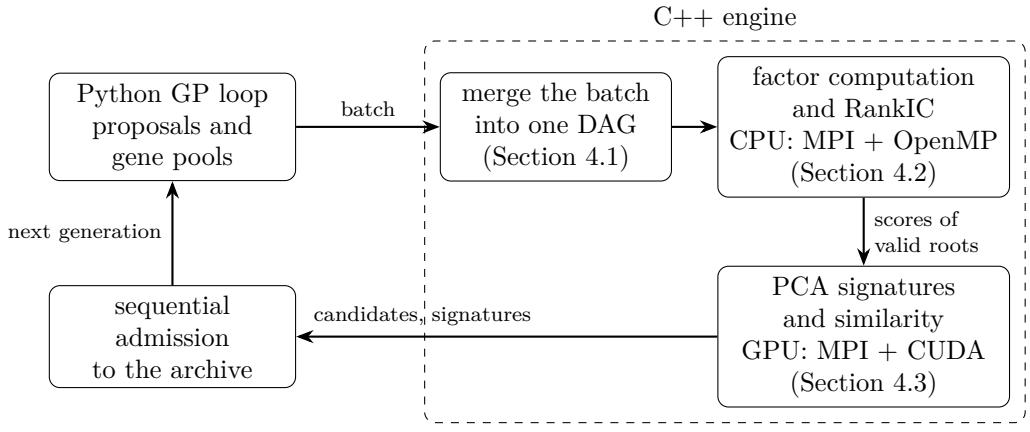


Figure 1: One generation of the miner. The engine receives a whole batch of expressions, evaluates it once as a DAG, and returns scores and signatures; admission stays sequential in Python.

Figure 1 shows one generation. The Python miner proposes a batch of expressions and hands the whole batch to the C++ engine. The engine merges it into one DAG (Section 4.1), computes

every node and the daily RankIC of every root on the CPU (Section 4.2), and computes the PCA signatures and similarities of the valid candidates on the GPU (Section 4.3). Admission to the archive compares each candidate with those admitted before it, so it stays sequential. The parallel model is therefore MPI + X: MPI splits the data over nodes, and X is OpenMP for factor computation and CUDA for the PCA part.

Each of the p ranks owns a contiguous block of about T/p days for all N stocks. With this choice:

- cross-sectional work (ranking, daily IC) is local;
- time-series operators need data from the preceding blocks;
- global statistics need reductions.

We decompose the data rather than the expressions. Each rank evaluates every node of the batch DAG (Section 4.1) level by level, but only on its own dates, so communication involves only a few boundary rows, never whole panels. Splitting by stock instead would make every cross-sectional rank a global sort; splitting by expression, as the Python version does, makes every process hold all data. A panel is stored as a contiguous float32 array in date-major order, together with the index $\texttt{lo} \leq 0$ of its first stored row; rows below 0 hold history that precedes the rank's own block.

4.1 The subexpression DAG and memory

A batch of expressions is merged into one DAG by *hash-consing*. Before a node is created, the key (operator, window, child ids) is looked up in a hash table. An existing node is reused, so a subexpression shared by several candidates is computed once. Normalisation is applied during construction:

- the children of commutative operators are ordered by id;
- a rank whose child is a rank returns the child;
- equal children of $-$ and $\div$ reject the whole expression, which is rolled back.

Children are always created before their parents, so the node array is already a topological order. Every rank builds the identical DAG, so node ids agree across ranks and halo messages need no descriptors.

For each node we record its *cumulative lookback* $h(n) = \ell(n) + \max_c h(c)$, where $\ell = w - 1$ for rolling windows and $\ell = k$ for lag, delta and pct_change, and its *halo rows* $\max_{\text{parents } q} \ell(q)$. Each node also counts its parents; when all parents are computed, its panel goes back to a buffer pool.

On the 1481 selected factors, merging all expressions into one DAG reduces 11,413 tree nodes to 4,052 distinct nodes, including leaves.

Memory model and scheduling. We model the lifetime of materialised panels as a red–blue pebble problem: a computed DAG value occupies limited fast storage, while eviction may require a transfer or recomputation later. This is a modelling framework, not an exact optimal scheduling algorithm. Our first-stage simulator computes each distinct node once and compares (i) level order with roots retained until the end, (ii) level order with roots consumed early, (iii) level order with operator layers split into chunks of C nodes, and (iv) a greedy topological order favouring nodes that immediately release the most input panels. Inputs remain live until their final consumer; repeated operand references, such as $a \times a$, are counted separately. A root that is also another root's input cannot be released after scoring alone. Chunk outputs are allocated together, so allocating a wide layer can determine the peak even with early release. The engine combines (ii) and (iii): roots are scored and released as soon as their chunk is done, and wide levels are processed in chunks (Section 4.2.6). Section 6.1 compares the four orders.

4.2 Factor computation and evaluation

4.2.1 Operator classes and their parallel axis

- element-wise operators sweep the array once, split over days;

- cross-sectional operators (rank, z-score) process one day (one row) at a time, split over days with dynamic scheduling;
- rolling operators process one stock (one column) at a time with $O(T)$ sliding-window updates, split over stocks; each thread copies the column into its own scratch array, computes, and writes back.

In the date-major layout a stock's history is a column with stride N floats (23.5 KB), so reading it touches one cache line per day for only four useful bytes. This is acceptable for two reasons. First, the loop over stocks uses a static schedule, so each thread processes a contiguous range of stocks: the T cache lines loaded for one column ($T \times 64\,\mathrm{B} \approx 93\,\mathrm{KB}$ for the full period, less per rank) stay in the 2 MB per-core L2 cache and serve the next 15 columns. Second, the gather is $O(T)$ per column, while the window kernels that follow do considerably more work per element (Kahan compensation, deque updates, sorted windows for the median), so the copy is a small part of the operator time. Storing panels stock-major instead would make the cross-sectional operators and the daily RankIC strided, and those are at least as expensive.

4.2.2 Calibrating floating-point differences

Small differences in floating-point rounding can change which windows look constant, and hence whether a correlation is finite, so we port the pandas 3.0.3 window kernels rather than textbook formulas.

- Rolling sums and means use Kahan summation with separate compensations for additions and removals. When the last n_{obs} observations are identical, the repeated value is returned exactly.
- Rolling variance uses Welford's update and recomputes the window from scratch when cancellation is detected ($m_2^{\mathrm{prev}} \cdot 10^3\varepsilon > m_2$).
- Windows count only non-missing observations. The minimum count is 1 for mean, sum and median, and $\min(3, w)$ for std, max, min, correlation and beta.
- Correlation and covariance use pairwise-complete observations. Beta divides by the variance of b alone.
- Infinite results of $+, -, \times, \div$, pct_change and beta become missing values. pct_change is computed in float32, as in pandas.

Max and min use monotone deques, and the median keeps the window sorted ($w \leq 60$).

4.2.3 Halo exchange versus redundant computation

A rolling operator on the first days of a block needs earlier values, which belong to the ranks on the left. We consider three strategies:

- **A, redundant computation:** each rank preloads H_0 extra days of raw data and recomputes subtrees on them, without communication;
- **B, halo exchange:** after each DAG level, each rank sends the rows its right-hand ranks need; all nodes of the level are packed into one message per partner (`MPI_Isend`/`MPI_Irecv`). When blocks are shorter than the halo, a rank receives from several ranks to its left (multi-hop halo);
- **Hybrid:** a subtree is recomputed if its cumulative lookback is at most H_0, and exchanged otherwise.

H_0 is therefore a single tuning knob: $H_0 = 0$ gives strategy B, and $H_0 \geq \max h$ gives strategy A. The redundant work per rank grows like $H_0/(T/p)$, while the number of messages grows with p when blocks get short, so the best H_0 depends on p and on whether neighbours share a node.

4.2.4 Recursive operator: ema

The exponential moving average follows pandas' `adjust=False` recursion with missing values: the previous weight decays by $(1-\alpha)$ every day, so after k missing days

$$y_t = \frac{o\, y_{\text{prev}} + \alpha x_t}{o + \alpha}, \qquad o = (1-\alpha)^{k+1}. \tag{3}$$

This is an affine function of y_{prev}, so a block of days maps its initial state through an affine map, and affine maps compose associatively. Each rank summarises its block per stock; the states are then passed along the ranks in a chain (rank r receives the state at its first day from rank $r-1$), and each rank runs its block from the correct initial state. The chain takes $p-1$ sequential hops per level that contains an ema, each carrying two numbers per stock. Rank r cannot start its ema until all ranks to its left have finished theirs, so the measured scan time includes this waiting: for the 10,000-expression batch it is 0.83 s at 72 ranks and 4.8 s at 288 ranks, against a total of 143 s and 75 s.

4.2.5 Daily RankIC and fitness

Daily RankIC is local to a rank, because a day's cross-section is never split. A root's RankIC is computed as soon as its DAG level is done, and the root's panel is then released like any other node. Each rank keeps only a small summary per root (segment sums and counts), and one `MPI_Allreduce` of all summaries gives the fitness of the whole batch.

4.2.6 Thread strategies and memory

Inside a rank we implement two thread strategies, selected at run time:

- **operator-level:** the nodes of a level are computed one after another, and each operator is split over days or stocks as in Section 4.2.1;
- **node-level:** the nodes of a level are distributed over the threads (`schedule(dynamic,1)`), and each node is computed by one thread. Nested parallel regions are disabled, so the operators run serially inside a thread.

Operator-level threading parallelises every node, including the narrow top levels of the DAG, but opens a parallel region per operator. Node-level threading has no per-operator overhead and keeps each node's data in one core's cache, but leaves threads idle when a level has fewer nodes than threads.

Threads only read shared input panels and write their own output panels, so no locks are needed on the data. The buffer pool and the per-operator timers are the only shared mutable state: output buffers are taken from the pool in a serial pass before the parallel loop, and the timers are updated in a critical section. The contention that remains is for memory bandwidth and cache capacity.

Memory handling went through three problems, each found by timing every operator separately on an earlier 32-core development server, and a fourth that appeared only with large batches:

- **Page faults on fresh memory.** At first every node allocated a new panel. The operating system only maps physical pages when they are first written, and it zeroes each page at that moment. With 32 threads writing new panels at once, these page faults queued in the kernel and took about 45% of the compute phase, outside the operators themselves.
- **Fix: a buffer pool.** Instead of returning a released panel to the operating system, the engine keeps the buffer, whose pages are already mapped, in a pool, and the next node that needs a panel takes the smallest pooled buffer that fits. Nothing is copied or moved; the memory simply stays allocated in RAM. The pool lives as long as the engine object, so it carries over from one batch to the next, as in a GP run where the engine is called generation after generation. Only the very first batch still pays for page faults, and the benchmark's warm-up run absorbs that. The pool doubled the single-rank 32-thread speed-up (10.9 to 19.9) and removed run-to-run variation of up to 50%.

- **Per-thread scratch arrays.** A rolling correlation used to allocate about 13 temporary arrays per stock, and many threads allocating at once contend inside the allocator. Each thread now allocates them once and reuses them for every stock, which cut 32-thread time by another 8%.
- **Wide levels.** A batch of 10,000 expressions has a level of 8614 nodes. Allocating all of them at once would need the output panels of the whole level in memory. The engine therefore processes a wide level in chunks of at most k nodes (`--chunk`): each chunk is allocated, computed, exchanged, scored and released before the next one. The result does not depend on k (checked against the serial engine with chunks of 250 and 300 nodes). With $k = 2000$, 72 ranks on one node peak at about 4.9 GB each, and one rank with 72 threads at 138 GB. Section 6.1 shows why chunking is needed: once roots are released early, the widest level sets the peak.

4.2.7 Communication and load model

For p ranks and a batch with exchanged node set $\mathcal{X}$:

- **Halo volume.** Each rank sends, for each exchanged node $n \in \mathcal{X}$, its last halo(n) rows of N floats, i.e. $4N \sum_{n \in \mathcal{X}} \text{halo}(n)$ bytes per rank, nearly independent of p. For the 10,000-expression batch of Section 6 this is 1811 of 12,083 nodes and 1.47–1.73 GB per rank (it grows slightly with p because short blocks forward rows from further left).
- **Messages.** One packed message per partner per level (per chunk, when a level is processed in chunks, Section 4.2.6). While a block is longer than the longest halo, the only partner is the left neighbour; when T/p drops below the halo, rows come from several ranks. For the 10,000-expression batch with chunks of 2000 nodes the measured maximum is 2 messages per rank up to 16 ranks and 4, 6, 12 and 24 at 36, 72, 144 and 288 ranks.
- **ema chain.** $p-1$ sequential hops per ema level, two numbers per stock each (Section 4.2.4).
- **Fitness.** One `MPI_Allreduce` of a short summary vector per root (segment sums and counts), under 0.5 ms at 288 ranks.

Per rank, computation shrinks as $1/p$ while the halo volume stays constant and the chain grows with p, so communication dominates at large p. In practice the synchronisation matters more than the volume: a rank cannot finish a level before it has received its left neighbour's rows for that level.

Load balance. Blocks have equal numbers of days, but not equal work. Rolling and element-wise operators process all N columns, but ranking and RankIC sort only the stocks with data, whose number grows from 2636 in 2016 to 4845 in 2021. The last block's cross-sectional work is therefore up to 1.8× the first block's, and the earlier ranks wait for it at every halo exchange and at the final reduction. Synthetic data have a constant number of stocks and isolate this effect.

4.3 PCA signatures and similarity on the GPU

Let $m = \lceil T/5 \rceil$ denote the number of sampled dates; here $m = 293$.

Sampling and ranking. PCA follows the factor engine's date partition. Each rank selects only its owned rows with global index $t \bmod 5 = 0$, excluding halo rows, so no date is counted twice. A GPU thread block sorts one sampled cross-section with bitonic sort, padding the stock count to a power of two; the current limit is 8192 columns. Ties receive average ranks, missing ranks are filled with 0.5, and each row is centred. Bitonic sorting costs $O(N \log^2 N)$, against $O(N \log N)$ for CPU comparison sorting; performance must be measured at the actual dimensions.

Batched power iteration and communication. Write the local date block as R_p. Every rank holds the same right vector v. The product $R_p v$ is local, while global normalisation and the

reverse product require

$$\|Rv\|_2^2 = \sum_p \|R_p v\|_2^2, \qquad R^\top u = \sum_p R_p^\top u_p. \tag{4}$$

A batch of B factors combines the reductions into B norms and BN vector components. With k iterations and a final Rv, there are $k+1$ norm reductions and k vector reductions. More ranks reduce local multiplication work but do not shorten the right-vector reduction. CUDA-aware MPI operates directly on device pointers, with support checked at startup. The solve supports FP32 or FP64; final signatures are stored as FP32, gathered, and assigned a consistent sign.

Tiled similarity GEMM and the cuBLAS comparison. Centre and normalise the candidate and archive signatures by row, giving $S_c \in \mathbb{R}^{B\times m}$ and $S_a \in \mathbb{R}^{A\times m}$. Then $C = |S_c S_a^\top|$, with element-wise absolute values, gives $B \times A$ absolute Pearson correlations. Here B is the candidate count, A is the current archive size, and m is the signature length; C_{ij} compares candidate i with archive member j. The handwritten CUDA kernel loads signature tiles into shared memory with 16×16 thread blocks, accumulates along the signature dimension, and fuses the absolute value. Candidate–candidate similarities are also precomputed in batches.

The independent cuBLAS reference uses identical device inputs, strict FP32 and disabled TF32. Its GEMM plus abs is compared with the handwritten kernel's fused operation. The library is 1.45–3.89 times faster; Table 11 gives the configurations and timings. cuBLAS is a performance reference only; the production backend retains the handwritten kernel.

Sequential admission and the serial fraction. After similarity precomputation, candidates are still admitted, replaced or rejected in their original order. FP32 estimates and conservative error bounds identify comparisons that could affect the nearest record, ties or threshold decisions; the CPU refines those comparisons in FP64 before updating the archive. If this sequential part occupies $f_{\mathrm{adm}} = T_{\mathrm{adm}}/T_{\mathrm{total}}$ of the workflow, accelerating everything else by a factor q cannot give a total speed-up above $[f_{\mathrm{adm}} + (1 - f_{\mathrm{adm}})/q]^{-1}$, with limit $1/f_{\mathrm{adm}}$. In the full-batch experiments of Table 10, sequential admission occupies about 1.48% and 2.64% for one and two GPUs, respectively. These fractions exclude similarity precomputation.

Data layouts and interfaces. The three layouts differ in how MPI ranks share work: jointly processing different dates of a factor, or processing complete factors separately. `date` retains the date partition and reduces each iteration; `transpose` first redistributes complete factors with Alltoallv, then solves locally; `factor` reads complete factors assigned to each rank. Besides a central archive, signature sharding is implemented: ranks compare their local records and use MAXLOC to select the global nearest one, while small metadata such as keys and quality remain replicated. GPU ranking also offers a NaN-preserving percentile interface and an FP64 entry point compatible with the original engine's double ranking for RankIC reuse. In the measured GP runs, factor values and RankIC are still computed on CPUs.

4.4 Power iteration versus Lanczos

Both methods use only products with R and $R^\top$, without explicitly forming $R^\top R$. Power iteration starts from a random unit vector with a fixed seed and alternates

$$u_j = \frac{Rv_{j-1}}{\|Rv_{j-1}\|_2}, \qquad v_j = \frac{R^\top u_j}{\|R^\top u_j\|_2}, \qquad s = \frac{Rv_k}{\|Rv_k\|_2}. \tag{5}$$

It retains only the current vectors. When $\sigma_1 > \sigma_2$ and the start has a component in the leading direction, secondary components decay approximately as $(\sigma_2/\sigma_1)^{2k}$. A smaller spectral gap slows convergence. Four steps are therefore the original program's budget, not a general convergence guarantee.

Lanczos also depends on singular-value gaps and the initial projection, but retains multiple orthogonal directions and solves in a growing subspace. For rectangular R, this implementation uses Golub–Kahan–Lanczos bidiagonalisation (Netlib description). It constructs left and right bases U_k, V_k and a small bidiagonal $B_k = U_k^\top R V_k$. If z is the leading left singular vector of B_k, the signature is approximated by $U_k z$. Both bases receive two passes of full reorthogonalisation. A handwritten Jacobi method solves $B_k B_k^\top$; breakdown stops expansion and retains the terminal contribution actually formed.

Extra storage is $O(m + N)$ for power iteration and $O(k(m + N) + k^2)$ for Lanczos, which also pays for orthogonalisation. A complete alternating update mainly requires one product with R and one with $R^\top$, plus normalisation or orthogonalisation. In this implementation k power iterations use $2k + 1$ products, whereas a normal k-step bidiagonalisation usually uses $2k - 1$; residual diagnostics add two more. The experiments compare iteration budgets, product counts and the cost of a common accuracy target. Power iteration supports CPU/MPI and CUDA; Lanczos currently runs on CPU/MPI.

Whether four steps suffice for admission must be checked through decisions, not just direction error. With the same initial vector and candidate order, eight rather than four steps change 637 actions and produce a final-membership symmetric difference of 372 (Table 14). Four steps therefore provide no general guarantee of exact admission decisions; the current budget is retained to reproduce the original program.

5 Validation

A faster engine is only useful if it computes the same numbers. We had a ready-made reference for this. An earlier full run of the original Python miner produced a factor library of 1481 factors, and Python had already computed their values, daily RankIC and fitness. Validation therefore means asking the C++ engine to compute the same things again and comparing them with Python's results item by item.

Parsing and DAG. We first checked that C++ reads expressions the way Python does. We parsed all 1481 library expressions plus 50 hand-written ones (at least one for every operator and every normalisation rule), 1531 in total, on both sides. Every valid expression gets the same depth and node count, and two expressions that Python considers equivalent (same normalised key) end up on the same DAG root in C++.

Operators. Comparing every cell of the full data would be slow, so we cut a sample of 120 days $\times$ 300 stocks from the real data, keeping all 132 fields, and evaluated 88 expressions on it. The first 48 test single operators, each with windows 1, 5 and 20, plus a few nested combinations; the other 40 are taken from the factor library, to check that operators still agree when combined as in real factors. The number of cases follows from this design, not from a target. A cell passes if $|q - p| \le 10^{-4}|p| + 10^{-5}\,\mathrm{rms}(p)$ and missing values are in the same places; the absolute term is there because pandas itself computes a few operators in float32.

With textbook sliding-window formulas only 53 of the 88 cases passed. Every failure came from rounding: a tiny rounding difference decides whether a window counts as constant, and that decides whether a correlation is finite, missing or infinite. After porting the pandas 3.0.3 window kernels line by line (Section 4.2.1) and treating windows that are constant within their range consistently, all 88 cases pass, with missing and infinite values in exactly the same places.

RankIC, fitness and signatures. Here two kinds of difference have to be told apart: a bug in the code, and rounding caused by storing panels in float32. We therefore compared everything twice. First we gave C++ pandas' own factor values, rounded to float32, so both sides had identical input and only the code was tested. The daily RankIC agreed in 88/88 cases (largest difference 8.9×10^{-16}), the fitness in 88/88 (8.3×10^{-17}), and the PCA signature was bit-identical to Python's

in 88/88. Then we let C++ compute everything from the raw data. 63 of 88 cases matched Python exactly; for the rest the daily IC differed by at most 1.5×10^{-4} and the fitness by at most 2.4×10^{-5}, and whether a factor is accepted agreed in all 88. These differences come from float32: two distinct float64 values can round to the same float32 and create a tie in the ranking. When we rounded pandas' own factor values to float32 and recomputed the IC in Python, we got exactly the same differences as C++, so the cause is storage precision, not the code. Signatures were also compared end to end on the full data (1461×5876): $1-$similarity was at most 6.9×10^{-6}, tiny next to the 0.9 correlation threshold used for admission.

Small hand-computed unit tests cover the corner cases that real data rarely hit: ties, perfectly positive and negative correlation, constant cross-sections, the segment split, the sign-consistency penalty and the three rejection rules. The ema scan is checked bit for bit against the serial ema for several block splits, missing-value patterns and spans.

Distributed against serial. The distributed engine is compared with our own serial engine rather than with Python. It must give the same validity for every candidate and fitness within 10^{-5}. We do not require bit-identical panels: rolling sums and variances restart at every block boundary, so the last bits of a cell can differ, and a cell close to zero can differ by a large relative amount without changing any ranking. In the runs where we counted them (up to 32 ranks), more than 99.998% of cells were bit-identical. Results for up to 288 ranks and both thread strategies are in Section 6.2.6.

6 Experiments

Workload. The factor-computation experiments evaluate one real batch from a run of the Python miner: the 10,000 distinct expressions of its depth-2 warm-start batch, which is the batch the engine would receive in that generation. Merged into one DAG they give 12,083 nodes in three levels (132 leaves, 3337 and 8614 operator nodes). Almost every non-leaf node is a root that has to be scored, so this batch is evaluation-heavy. Unless stated otherwise each run evaluates the whole batch on the full real data, $T = 1461$ days and $N = 5876$ stocks.

Timing. Every run is timed with `MPI_Barrier` followed by `MPI_Wtime`. Each phase (loading, operators, halo exchange, ema scan, RankIC, waiting, reduction) is the maximum over ranks, and the total is the slowest rank's wall time. Most configurations run once to warm up (which also fills the buffer pool, as in a GP run where the engine is called generation after generation) and then three times, and we report the median. The slow ones (one rank, and two or four ranks or threads, 21 to 84 minutes each) run once without warm-up; for them the page faults of the first run are a small part of the time. Wide levels are processed in chunks of 1000 nodes on one node and 2000 nodes on several nodes (Section 4.2.6). Loading is reported separately because it depends on the shared file system. All plots use logarithmic axes for time, problem size and core count. Runs used Intel Xeon 8452Y nodes only; each job records the CPU model of its hosts.

6.1 DAG and memory

Inputs and measurement scope. S1 and S2 use random proposal batches, not consecutive evolved GP populations. The depth-3 batches share one randomly generated gene pool; this preserves reusable building blocks but does not establish real evolutionary reuse. Graph analyses run on the user's arm64 Mac (Python 3.13.5). The serial planner benchmark warms up once and measures three times; construction includes validation, normalisation and adjacency construction, while scheduling includes reference and live-panel checks. Neither time includes factor evaluation. The Vanda validation below is measured separately with GNU time. No MPI speed-up is inferred from these planner measurements.

6.1.1 Subexpression sharing and serial planning

Batch	Expressions	Tree nodes	DAG nodes	Reduction	Operator-only
Depth 1	10,000	24,157	10,000	58.6%	0.0%
Depth 2	10,000	35,022	11,591	66.9%	39.3%
Depth 3	9,991	40,942	13,135	67.9%	44.5%
Depth 3 warm start	29,955	125,465	32,825	73.8%	54.4%

Table 1: Random-batch sharing. Tree counts precede normalisation and include leaves; operator-only reduction excludes leaves. Neither reduction is a measured speed-up.

Depth 1 shares only field leaves in these batches: its operator count is unchanged. The warm-start DAG has 132 leaves and 7,117, 15,273 and 10,303 nodes at levels 1–3. The widest layer is therefore not the number of roots or the peak live set. Python and C++ agree on canonical node sets, levels, subtree sizes, reference counts, accepted inputs and root order for all measured prefixes.

Expressions	V_d	E_d	Build (s)	Chunk 256 (s)	Greedy (s)
1,000	2,190	2,732	0.0532	0.00148	0.0209
5,000	7,805	10,171	0.2490	0.00533	0.1581
10,000	13,210	17,377	0.4730	0.00898	0.3205
29,955	32,825	44,032	1.4021	0.02312	1.2302

Table 2: Serial Python planning on warm-start prefixes: distinct vertices V_d and child–parent edges E_d. The last two columns time scheduling only.

The largest graph has 44,034 operand references but 44,032 distinct edges, because repeated inputs count twice for lifetime management. Both graph dimensions grow in this experiment; they are not independently controlled. Construction grows approximately with batch size over these four points, while greedy scheduling is more expensive than simple chunk scheduling. These are planner overheads, not C++ factor runtimes, and no same-hardware end-to-end overhead ratio has been measured.

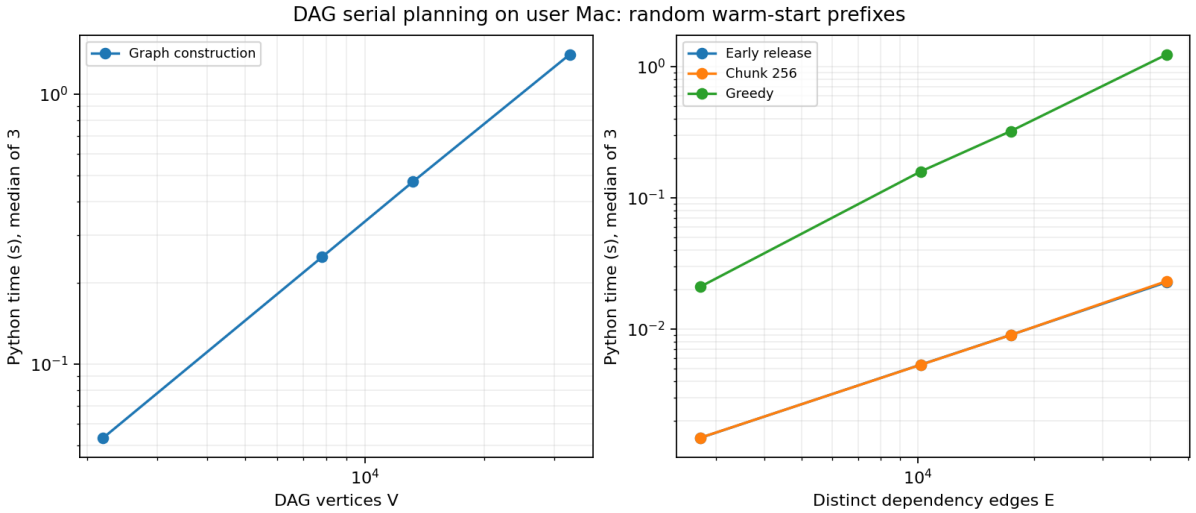


Figure 2: Planner construction versus vertices and scheduling versus edges, on log–log axes. Mac measurements are not combined with Vanda engine timings.

6.1.2 Peak memory against batch size

For equal float32 panels, the model converts a peak live count L into $4LN\lceil T/p \rceil$ bytes, with $T = 1461$, $N = 5876$ and $p = 1$ here. This excludes halos, retained pool buffers, target ranks, operator scratch and communication/PCA workspaces; it is not RSS. Operator-group counts describe scheduling granularity, not measured MPI synchronisations or messages.

Schedule, 29,955 expressions	Peak panels	Panel GiB	Groups (proxy)
Retain roots	32,290	1,032.67	3
Early release	17,730	567.02	3
Chunk 1,024	4,850	155.11	33
Chunk 256	4,086	130.67	129
Chunk 64	3,896	124.60	512
Greedy	1,553	49.67	32,693

Table 3: Logical live-panel peaks on the random warm-start batch.

Early release reduces the peak by 45.1% relative to retained roots; chunk 256 reduces it by another 77.0%. Reducing the chunk from 256 to 64 saves only 4.65% more panels but nearly quadruples groups. Greedy uses the fewest panels and permits on-demand field loading, but its fine-grained execution may reduce parallelism. This is a memory–granularity trade-off, not evidence that greedy minimises runtime.

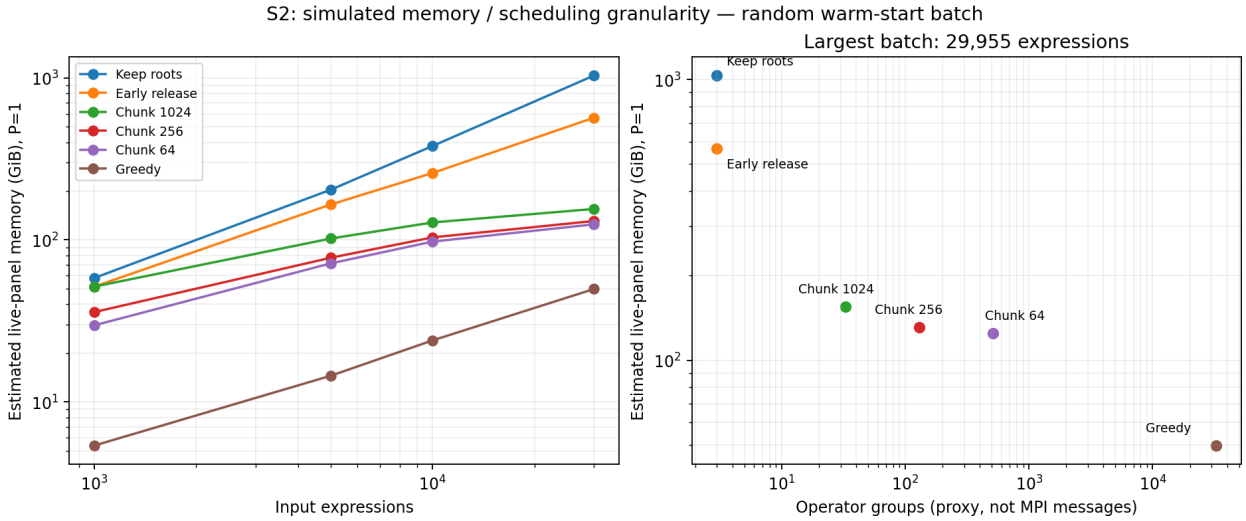


Figure 3: Simulated memory against batch size (left) and scheduling groups for the largest batch (right), with logarithmic axes. Groups are not MPI message counts.

Small engine check (S4). On Vanda CPU node CN-016, the handed-off serial C++ exporter evaluates the same first 200 expressions on 120 days × 300 stocks. OpenMP, MPI and CUDA are disabled (GCC 14.3.0, Release). Formula chunks of 16, 64 and 200 are each run three times in that order with GNU time. Each chunk constructs a separate DAG; this differs from dividing a layer of one shared DAG.

All nine runs exit successfully. A server-side comparison records 200 identical score records per run against the chunk-200 reference, excluding only chunk/slot metadata; RankIC, fitness and segment statistics agree. The retained verification record and logs were reviewed; raw score files were not included in the review archive. Chunk 64 uses 26.4% less median RSS than chunk 200. CPU user time is about 0.88 s for every setting, wall times overlap, and file-system caching and fixed run order were not controlled, so no runtime winner is claimed. The exporter suppresses R

Formula chunk	Median wall time (s)	Range (s)	Median RSS (MiB)
16	1.60	1.53–1.93	25.42
64	1.39	1.23–1.58	25.42
200	1.29	1.26–1.75	34.53

Table 4: Actual single-node formula-chunk experiment; three separate processes per setting.

output but still performs signature-row preprocessing. This checks the memory trend of formula subdivision, not exact S2 scheduling.

Consequences for the engine. These results fix how the engine evaluates a batch in the experiments that follow. Sharing (Table 1) is why a whole batch is merged into one DAG before evaluation: the real batch of Section 6 gives 12,083 DAG nodes for 10,000 expressions. Once roots are released early, the widest level sets the peak (Table 3), so the engine also processes wide levels in chunks (Section 4.2.6): the widest level of that batch has 8614 nodes, and the runs below use chunks of 1000 nodes on one node and 2000 on several. Chunking does not change any score (Table 4 and Section 4.2.6). Its cost is one halo exchange per chunk instead of one per level; we did not compare run times with and without chunking.

6.2 Factor computation and evaluation

6.2.1 Serial run time against problem size

One rank and one thread. We vary one dimension at a time: the number of days T (prefixes of the real data), the number of stocks N (the first N stocks), and the batch size (the first E expressions of the batch). Synthetic data extend T and N beyond the real data; there we use the first 500 expressions, because eight times the days with more expressions does not fit in a node's memory (Figure 4).

dimension	range	total time (s)	slope
T, real, 10,000 expressions	91 – 1461	245 – 5059	1.09
N, real, 10,000 expressions	367 – 5876	376 – 5059	0.95
T, synthetic, 500 expressions	1461 – 11688	540 – 4825	1.05
N, synthetic, 500 expressions	5876 – 23504	540 – 2304	1.05
batch size, real	625 – 10,000	568 – 5059	0.89

Table 5: Serial run time. The slope is fitted on log–log axes to the time without loading.

The whole batch takes 5059 s (1.4 hours) on one core, which is why a generation of the Python miner is slow. The run time is close to linear in every dimension, as Section 1.7 predicts: operators are $O(TN)$ per node and RankIC is $O(TN \log N)$, which shows up as the slope slightly above 1 on synthetic data. The slope in T on real data (1.09) is larger because the market grows: the early years, which the short prefixes cover, have fewer listed stocks per day. The slope in batch size is below 1 (0.89) because larger batches share more subexpressions: 625 expressions need 2.1 DAG nodes each, 10,000 need 1.2. RankIC takes 70% of the serial time (3522 s against 1536 s for the operators), because 83% of the nodes of this batch are roots and each root's RankIC sorts every day's cross-section.

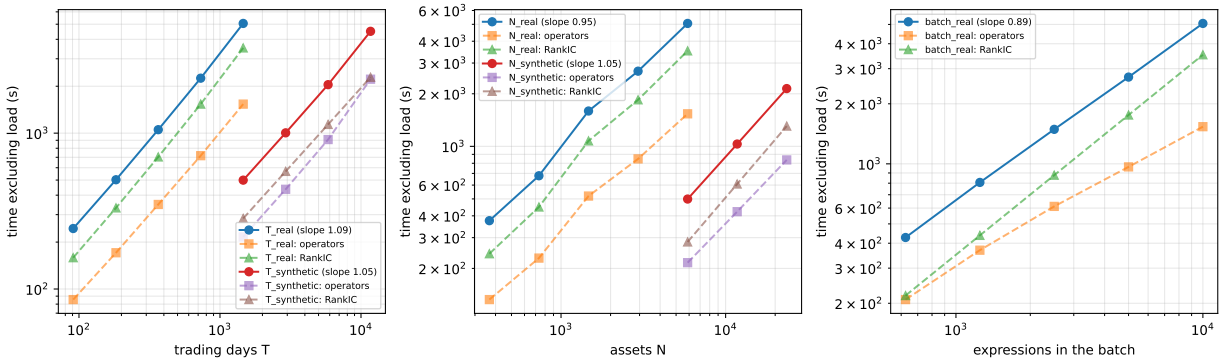


Figure 4: Serial run time against T, N and batch size, with the operator and RankIC parts. Real data: the 10,000-expression batch or its prefixes; synthetic data: the first 500 expressions.

6.2.2 Strong scaling (pure MPI)

One thread per rank, $H_0 = 0$, 1 to 72 ranks on one node and 144 and 288 ranks on two and four nodes (Table 6, Figure 5).

ranks	1	2	4	8	16	36	72	144	288
nodes	1	1	1	1	1	1	1	2	4
time (s)	5060	2825	1534	815.7	436.4	241.3	142.7	94.1	75.0
speed-up	1	1.79	3.30	6.20	11.6	21.0	35.5	53.8	67.5
efficiency	1	0.90	0.82	0.78	0.72	0.58	0.49	0.37	0.23

Table 6: Strong scaling, 10,000 expressions on the real data.

The phases scale very differently. RankIC falls from 3519 s to 18.5 s at 288 ranks (190 times), because it is purely local. The operators fall from 1539 s to 66 s at 72 ranks but only to 52 s at 288 ranks, and the halo phase stays at 22–24 s from 16 ranks on. At 288 ranks each block has about 5 days, while the exchanged nodes need up to 89 days of history: each rank receives 1.7 GB of halo rows, more than its own share of the panels, and the rolling windows restart at every block boundary, so per-rank operator work no longer shrinks with p. Waiting is large at every rank count (451 s of 2825 s at two ranks), consistent with the uneven blocks of Section 1.7: the second half of the period has more listed stocks. The Karp–Flatt serial fraction falls from 0.12 at two ranks to 0.011 at 288 and does not grow, so the loss is not a growing serial part but a per-rank cost that does not shrink: halo, boundary restarts and imbalance. Per-rank timing to separate these is future work.

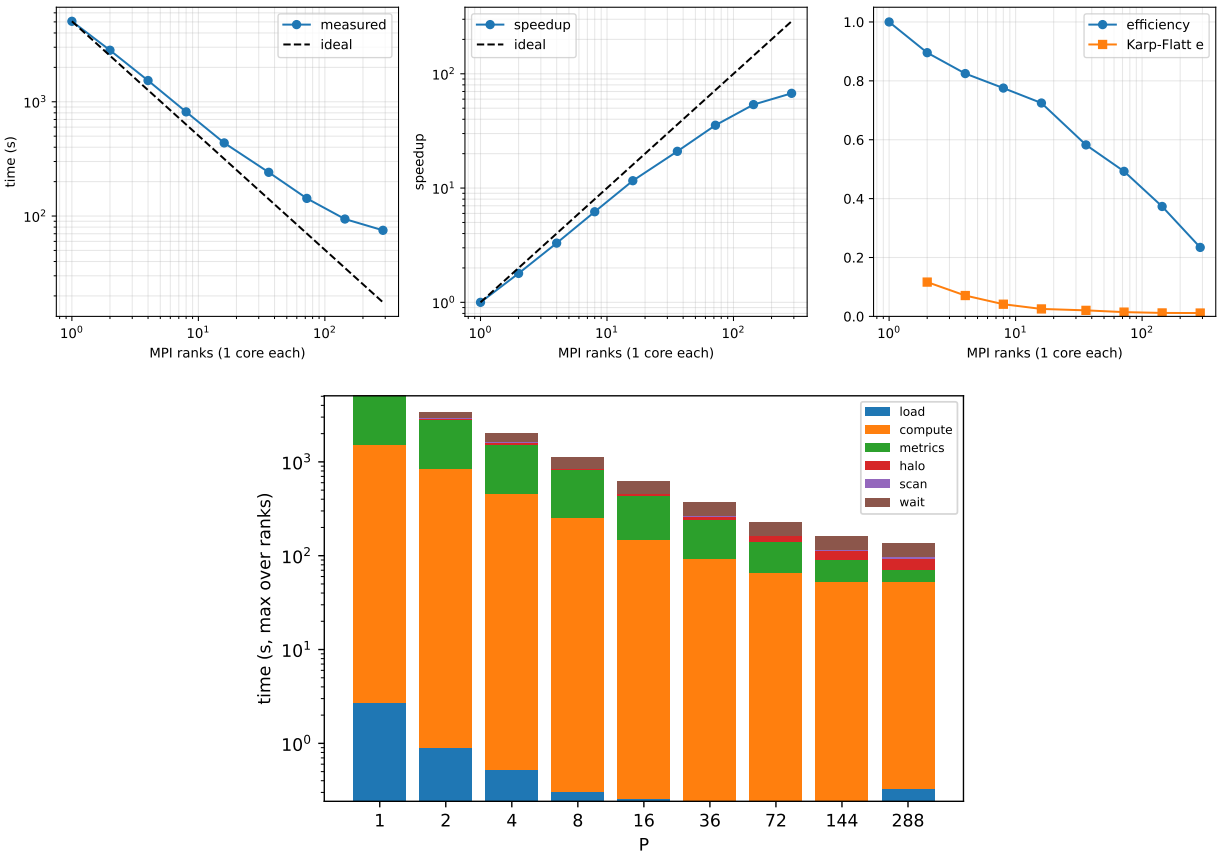


Figure 5: Strong scaling: time, speed-up, efficiency and Karp–Flatt metric (top); time per phase (bottom).

6.2.3 Weak scaling (pure MPI)

Synthetic data with 64 days per rank (from $T = 64$ at one rank to $T = 18432$ at 288 ranks), $N = 5876$, the first 1000 expressions of the batch (1956 DAG nodes); with the whole batch, 72 ranks with 64 days each would not fit in one node's memory.

ranks	1	2–16	36	72	144	288
time (s)	40.4	≈54.0	61.7	61.9	62.1	63.7
efficiency	1	0.75	0.66	0.65	0.65	0.63

Table 7: Weak scaling on synthetic data, 1000 expressions.

The drop from one to two ranks is the halo exchange (about 8.5 s, rising only to 9.9 s from 36 ranks on) and the matching boundary work. The efficiency falls again between 16 and 36 ranks, when the node fills up and its memory bandwidth is shared, and then stays at 0.63–0.65 across two and four nodes, so crossing the network adds little (Table 7, Figure 6). Synthetic data are used here because the real data cannot grow with the number of ranks; their blocks also have equal work, which separates communication from imbalance.

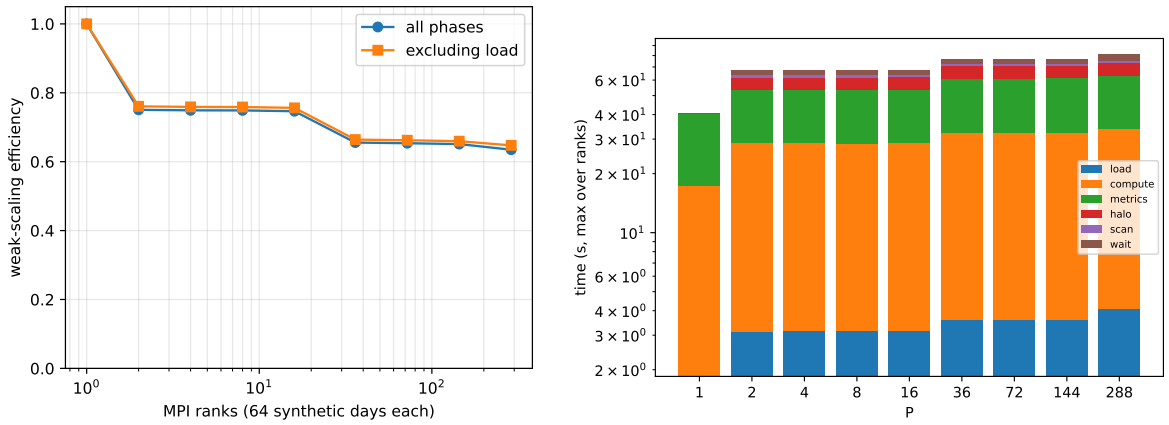


Figure 6: Weak scaling efficiency and time per phase.

6.2.4 Communication

Halo exchange versus redundant computation. Raising H_0 replaces exchanged nodes by recomputation. With pure MPI, $H_0 = 10$ changes almost nothing (1811 to 1591 exchanged nodes); $H_0 = 30$ leaves 715, and $H_0 \geq 60$ leaves 53 nodes and 40–45 MB of halo per rank instead of 1.5–1.7 GB, at the cost of 60,944 redundant rows per rank. The total time nevertheless rises: from 142.7 to 159.1 s at 72 ranks (+11%), from 94.2 to 109.7 s at 144 (+16%) and from 72.0 to 86.7 s at 288 (+20%) (Figure 7, left). The halo time also rises (23.7 to 31.1 s at 72 ranks) although 97% less data is sent, so it is not transfer time but time spent waiting for the left neighbour to finish the level, which the redundant work makes longer. For this batch exchanging everything ($H_0 = 0$) is best; with hybrid splits the picture is the same (72 ranks $\times$ 4 threads: 50.8 s at $H_0 = 0$, 54 s at $H_0 \geq 60$). With fewer ranks per node H_0 hardly matters: 16 $\times$ 18 stays at 43.0–43.9 s (43.1 s at $H_0 = 0$), and 4 $\times$ 72 varies between 48.2 and 52.4 s against 55.0 s at $H_0 = 0$, without a trend in H_0; with one rank per node there are only three exchanging pairs, so there is little to save.

Shared memory versus the network. The same number of ranks packed on as few nodes as possible or spread over four nodes: 36 ranks take 241.3 s on one node and 212.3 s on four; 72 ranks take 142.7 s and 126.4 s; 144 ranks take 94.1 s on two nodes and 94.0 s on four (Figure 7, right). Spreading is faster: the extra network hops cost less than sharing one node's memory bandwidth among more ranks.

Which parts communicate most. The halo dominates by far: each rank sends 1.5–1.7 GB per batch, so 288 ranks move about 500 GB in total, against 6 GB for four ranks. The ema chain needs $p - 1$ sequential hops with two numbers per stock; including the waiting it causes it takes 0.83 s at 72 ranks and 4.8 s at 288. The fitness needs a single `MPI_Allreduce` of the IC summaries, under 8 ms even at 288 ranks.

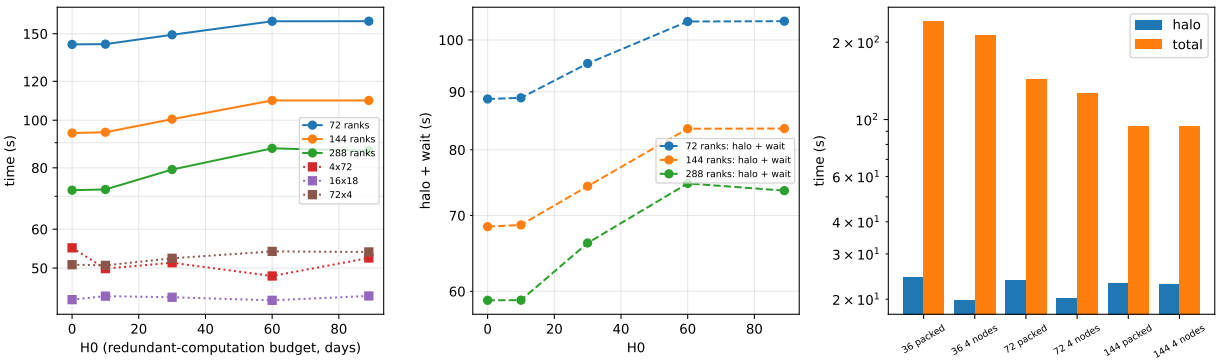


Figure 7: Left: total time against H_0 for pure MPI (solid) and hybrid splits (dotted). Middle: halo plus waiting time. Right: the same rank count packed and spread over four nodes.

6.2.5 MPI + OpenMP

One rank. With one rank and 2 to 72 threads, operator-level threading takes 2613, 1349, 690.9, 360.9, 200.2 and 110.3 s, and node-level threading 2528, 1267, 635.9, 321.9, 166.6 and 89.2 s (Figure 8, left). Node-level threading is faster at every thread count and reaches a speed-up of 56.7 at 72 threads (operator-level: 45.9). With 3337 and 8614 nodes in the two operator levels there is always enough work for every thread, and no parallel region is opened per operator. On one node, 72 threads (89.2 s) are 1.6 times faster than 72 ranks (142.7 s), because they need no halo exchange and no boundary restarts.

288 cores. We split four nodes into ranks × threads (Table 8, Figure 8, right).

ranks × threads	288×1	144×2	72×4	36×8	16×18	8×36	4×72
operator-level (s)	75.0	61.4	50.8	47.5	43.1	43.6	55.0
node-level (s)	75.0	48.9	37.7	32.8	29.3	28.1	**27.8**

Table 8: 288 cores split into MPI ranks and OpenMP threads, 10,000 expressions.

With node-level threads the time falls steadily as ranks are replaced by threads, and the best split is 4 ranks × 72 threads (one rank per node): 27.8 s, 2.7 times faster than pure MPI on the same 288 cores and 182 times faster than one core; 8 × 36 (28.1 s) and 16 × 18 (29.3 s) are nearly as fast. Each rank still receives about 1.5 GB of halo, so the total halo traffic is proportional to the number of ranks, and the halo phase falls from 22 s at 288 ranks to 1.5 s at four. Operator-level threading is slower on every split; most of the gap is in evaluation, where RankIC stays at about 29 s on every split against 18 s with node-level threads. With 72 threads per rank operator-level threading loses again (55.0 s): each operator opens a parallel region over 72 cores on two sockets.

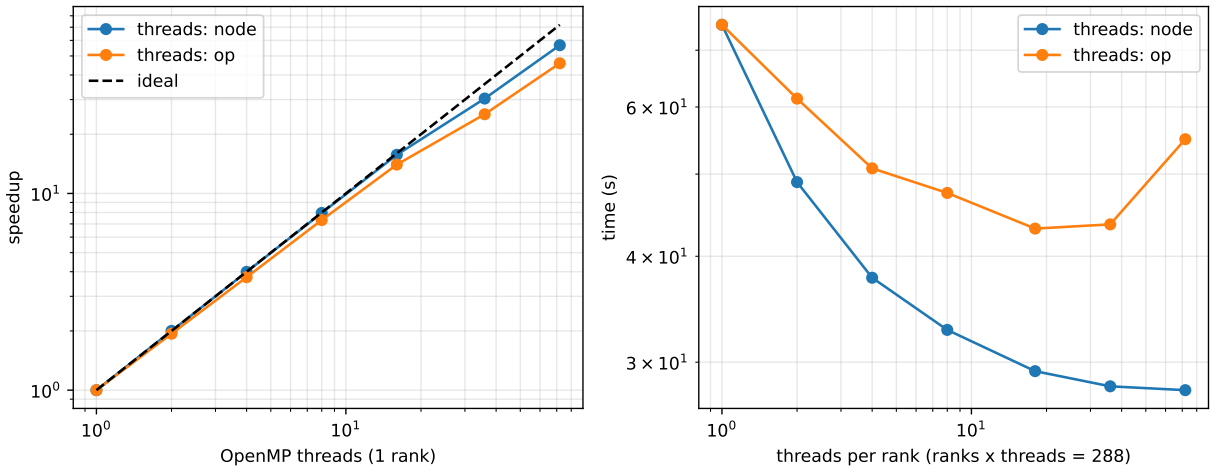


Figure 8: Left: speed-up of one rank with OpenMP threads. Right: 288 cores split into ranks × threads, with both thread strategies.

6.2.6 Consistency at scale

The distributed result is compared with the serial one with the criterion of Section 5 (the same candidate validity and $|\Delta\text{fitness}| \leq 10^{-5}$). The check keeps every root's result and also runs the serial engine on rank 0, so it uses the first 1000 expressions of the batch. It passes for 72 ranks on one node, 288 ranks on four nodes, and 16 ranks × 18 threads with node-level threads on four nodes, all with chunked levels, and for 16 ranks and 4 ranks × 4 threads with chunks of 300 and 250 nodes.

6.3 PCA signatures and similarity

6.3.1 Serial cost

The PCA input is constructed separately: random depth-3 proposals produce 9388 nonconstant matrices R, of which 8866 candidates are eligible for QD; four-step power iteration gives 2988 archive members. The lower-depth gene pool also comes from random proposals. This input differs from the earlier fixed depth-2 batch of 10,000 expressions. We first vary the size of 128 real panels, then report performance on the full R batch.

The original engine first computes 128 factor panels. Their values are held fixed while changing the sampling interval (1, 2, 5, 10, 20), stock prefix (367, 734, 1469, 2938, 5876), batch size B and iteration count k, giving 19 configurations. Each has one warm-up and three timed repeats, including fresh ranking, signatures and QD. Eight-thread factor generation takes 86.410 s and is reported separately. PCA is single-threaded and includes sample copying, ranking, solving and admission; serialisation is separate.

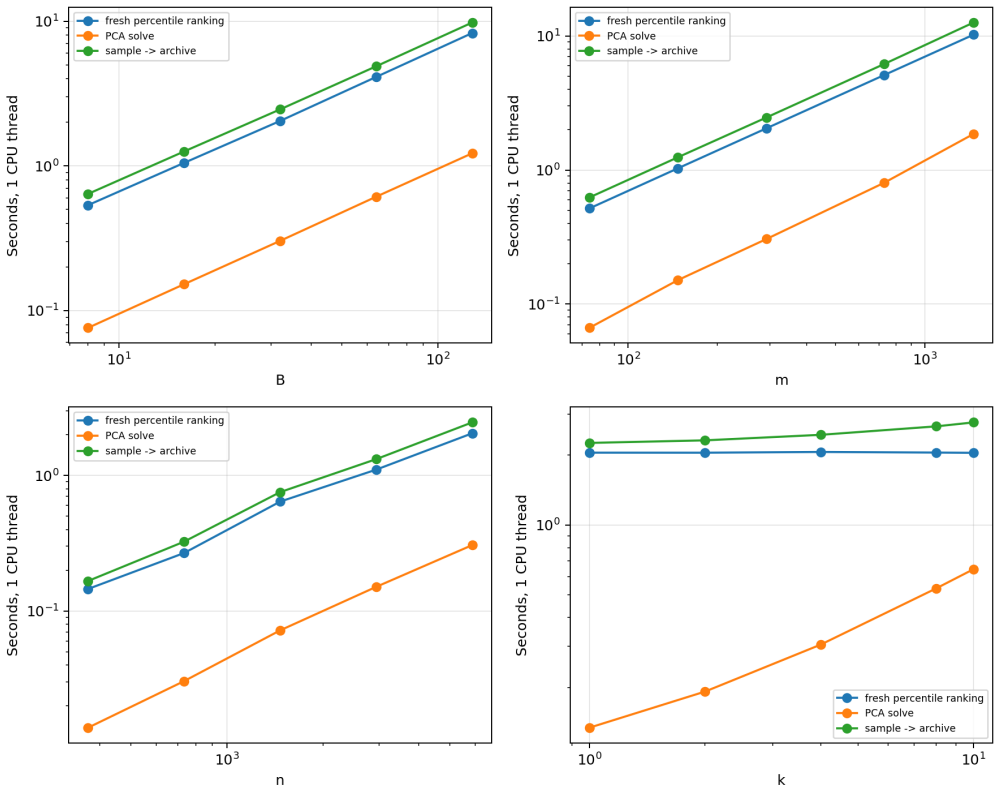


Figure 9: PCA size experiments on 128 real panels, varying B, sampled dates m, stocks N and iteration count k after fixing panel values.

In Figure 9, time grows with batch size, sampled dates and stocks. More iterations mainly affect the solve; ranking and preparation remain substantial. Matrix–vector work is approximately $O(BmNk)$, and synthetic dimension experiments also show near-linear growth in B, m, N. The stock-prefix experiment does not recompute cross-sectional operators or RankIC on a smaller universe: it measures PCA input-size changes, not the cost of mining a smaller stock universe.

6.3.2 GPU kernels against the CPU

Table 9 measures ranking, iterative multiplication and similarity separately. FP32/FP64 denotes iteration precision; ranking and coarse similarity both use FP32. Ranking is substantially more expensive than the two multiplication stages. FP64 nearly doubles iterative multiplication time but leaves ranking unchanged, so the whole workflow does not slow by the same factor.

Table 9: GPU component timing in ms: one warm-up, then the median of three repeats.

Precision	Batch B	Ranking	Matvecs	Similarity
FP32	8	21.212	0.964	0.097
FP32	32	84.694	3.737	0.156
FP32	64	169.285	7.450	0.273
FP64	8	21.209	1.773	0.098
FP64	32	84.694	7.023	0.157
FP64	64	169.289	14.017	0.272

For all 9388 preconstructed matrices R, 72 CPU cores with the `date` layout take 13.366 s, one A40 takes 64.766 s, and two A40s on one node take 36.713 s (Figure 10). This includes reading, preparation, transfers, computation, similarity, admission and signature writes inside the loop, but excludes factor generation. MPI startup, GPU initialisation and final JSON serialisation are

separate; writes do not include an fsync durability guarantee. The input is already ranked, so this total excludes GPU sorting. CPU uses 72 ranks, FP64 and batch 576; GPU uses 1/2 ranks, FP32 and batch 128. These are comparisons of actual resource and implementation configurations, not an experiment changing only the device.

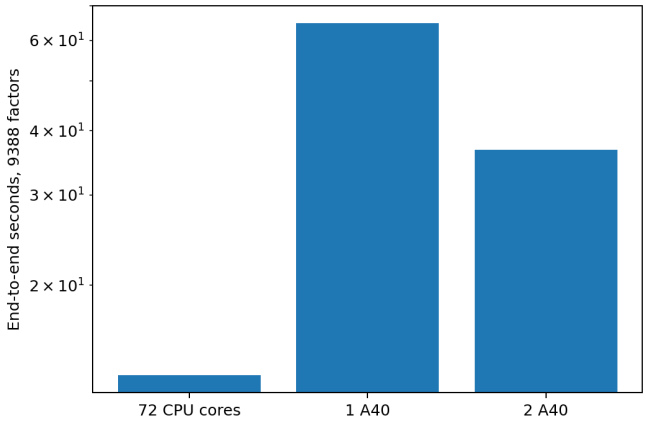


Figure 10: PCA workflow time for the same 9388 matrices R, using the stated CPU and GPU configurations.

Table 10 shows that 72 CPU processes share reading, whereas the GPU paths have only 1/2 host processes. The single-GPU pure GEMV median is 1.090 s, much less than the full workflow. Reading, host preparation, upload and synchronisation need joint optimisation. The `solve` stage includes GPU preparation and computation, with nested timers; a slower workflow does not imply slower GPU multiplication.

Table 10: Configurations and median stage times for already-ranked R, in s. The solve contains preparation and nested stages; kernel and communication timers must not be added to it.

Configuration	Ranks	Precision	Batch B	Read	Solve	Total
CPU	72	FP64	576	1.496	11.537	13.366
1 A40	1	FP32	128	34.170	27.429	64.766
2 A40	2	FP32	128	18.845	17.479	36.713

Sequential admission timing includes FP64 refinement and archive-state maintenance, excluding similarity precomputation. Compute $T_{\text{adm}}/T_{\text{total}}$ per repeat and then take the median of three repeats: the fractions are 8.12%, 1.48% and 2.64% for 72 CPU cores, one A40 and two A40s. These fractions describe this PCA workflow; the full GP serial fraction requires full-loop timing.

The independent cuBLAS reference uses identical device inputs, strict FP32 and disabled TF32 (cuBLAS documentation). Each block executes 50 operations; three blocks give the median, excluding warm-up, transfers, allocation and normalisation. Because the handwritten kernel fuses the absolute value, Table 11 compares it with library GEMM plus abs, with pure GEMM also shown. Here abs means element-wise absolute value. The speed-up is handwritten time divided by library GEMM+abs time; values above 1 favour the library. cuBLAS is 1.45–3.89 times faster, leaving room to improve handwritten multiplication. This ratio cannot directly be converted into a full-GP speed-up. cuBLAS remains a reference only; production targets use handwritten code.

Table 11: Independent similarity comparison with cuBLAS; times are in μs.

B	Archive A	Handwritten	cuBLAS GEMM	GEMM+abs	Speed-up
128	50000	1465.10	282.17	376.59	3.89
32	16384	121.53	47.66	51.45	2.36
64	16384	230.63	53.35	62.32	3.70
8	16384	66.31	43.72	45.81	1.45

6.3.3 MPI + CUDA scaling

Figure 11 shows CPU strong scaling for the three layouts. More ranks help at low counts, then gains flatten and some configurations become slower. Per-iteration vector reduction in `date` does not shrink with rank count; `transpose` pays redistribution costs, and `factor` depends on parallel reading. Weak-scaling time still grows and fluctuates when the equivalent complete-matrix workload per rank is held fixed; efficiency does not remain near 1. Occasional superlinear points need to be interpreted with node and run conditions.

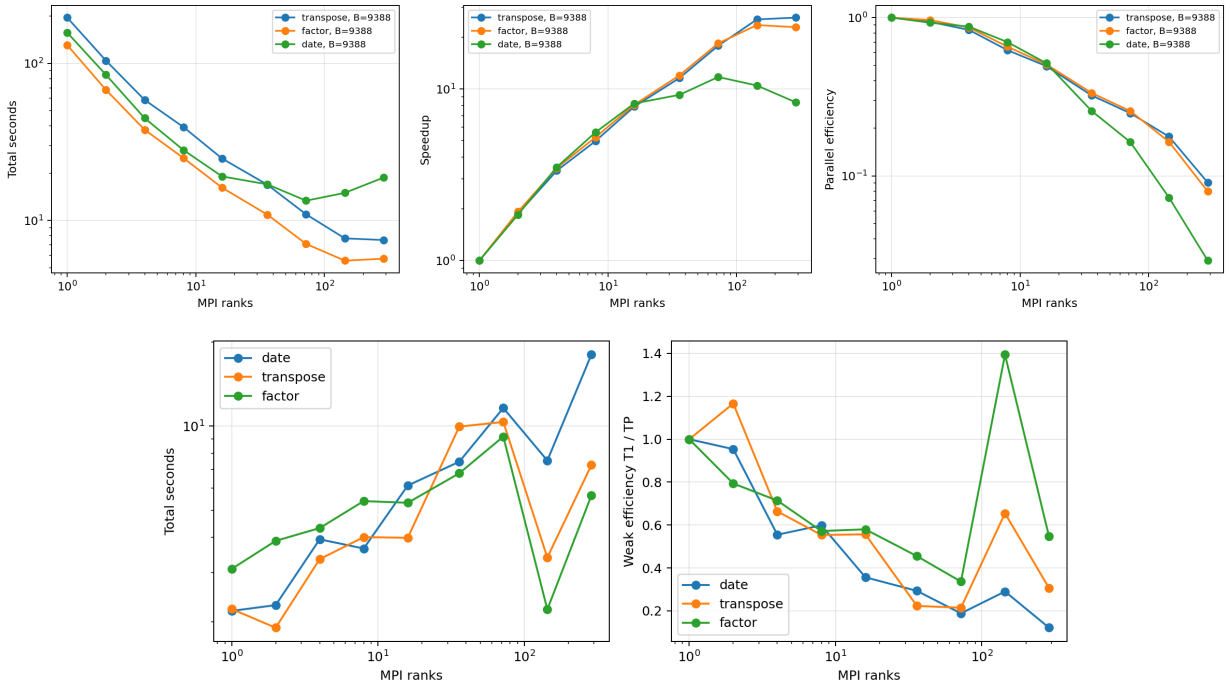


Figure 11: CPU strong scaling (top) and weak scaling (bottom) of the PCA module. Layouts differ in reading, preparation and communication; comparisons use their own total times.

Two A40s on one node are 1.764 times faster than one. Date blocks are split between the GPUs, with norms and right vectors merged by device-memory Allreduce. A separate two-node, one-GPU-per-node correctness check does not establish cross-node speed-up. Archive-scale experiments fix 128 candidates: signature sharding reduces time for the tested large archives, but each candidate still needs MAXLOC and a sequential decision, limiting scaling.

6.3.4 Agreement of signatures and admission decisions

In the full-batch FP32/FP64 comparison, no admission actions change. Every triple of action, nearest key and replaced key agrees, and the final-membership symmetric difference is zero. Replaying supplied signatures and recomputing from the same FP32 R also agree. This supports

implementation and precision choices for this input; changes to the solver, iteration budget and initial vector are examined separately below.

Backend and archive-state checks. Given identical signatures, candidate actions, nearest keys, replaced keys and final members agree with the original Python reference for CPU, CUDA and MPI-sharded archives. Boundary cases cover values near 0.90, tied maxima, identical keys, a quality increment exactly equal to 0.002, and invalid signatures. One complete batch and two batches joined through an archive snapshot give the same final state. Actual GP checks also cover interruption/resume and backend identity. Direction error and the admission threshold are different quantities; archive agreement is established by decision-by-decision checks.

Actual GP integration and 16-core calibration. An initial interface check uses 120 dates, 300 stocks and 6 fields, with one generation at each of three depths. CPU/MPI and CUDA both retain 24 archive members, with GP walltimes 189.463 s and 268.381 s. The full data dimensions are then used: 1461 dates, 5876 stocks and 132 fields, population 64, warm-start multiplier 3, gene-pool limit 32, and one generation at each depth 1, 2 and 3. Windows are 1/5/10/15/20/30/60, sampling is every fifth date, and power iteration uses four FP64 steps. Table 12 reports this configuration.

Table 12: Three-depth GP integration at full data dimensions. Setup and GP walltime are timed separately; the shared-CPU row is a later independent run.

Backend	Ranks	Threads/rank	Setup / s	GP wall / s	Evaluated	Archived
CPU	16	1	371.334	233.940	1450	325
CUDA (1 A40)	1	16	0.046	336.705	1450	325
CPU (shared)	16	1	0.052	234.358	1450	325

CPU uses 16 single-threaded MPI ranks. CUDA uses one process, 16 CPU threads and one A40, with an allocation of 36 cores and 240 GB. Factor values and RankIC remain on CPUs in both paths; GPU handles ranking, PCA and similarity. The first CPU run profiles the fields; CUDA and the later shared-CPU run reuse an identity-checked field cache, so setup times are not algorithm speed comparisons. GP walltime includes MPI subprocess startup, the factor engine, PCA, QD, Python control and checkpoints.

The CPU and CUDA runs finish with the same 325 keys but are not bitwise identical: maximum absolute quality difference among common members is 4.70977×10^{-5}, and maximum FP32 signature-component difference is 0.00370505. Evaluations include warm starts and searches at all depths, so the count is not population times generations. The archive accumulates across batches and may exceed one generation's population.

The later shared-resource calibration allocates 16 cores and 112 GB on CN-161 (Xeon Platinum 8452Y). The same three-depth configuration takes 234.358 s and retains 325 members. A separate fixed, deduplicated batch of 1000 actual expressions goes through factors, RankIC, PCA and QD in the same order. One warm-up and three measurements give a median batch walltime of 64.648 s, range 64.308–65.301 s. Each run is checked against original Python QD decisions and final members; repeated numerical outputs within the same layout are bitwise identical. Input validation is separately timed at 102.041 s.

The calibration job includes these repeated batches and the three-depth GP, taking 10 minutes 28 seconds in total. The node is shared with other jobs. These measurements validate the resource route and integration, rather than forming a speed-up ratio against dedicated nodes; they also differ in input and timing scope from the fixed 10,000-expression batch of Section 6.2.1.

6.4 Power iteration versus Lanczos

6.4.1 Accuracy against iterations

For the leading left singular vector u_{ref} from a full NumPy SVD, define direction error as

$$e = 1 - \frac{|s^\top u_{\text{ref}}|}{\|s\|_2 \, \|u_{\text{ref}}\|_2}. \tag{6}$$

The absolute value removes the sign ambiguity. The strict target is $e \leq 10^{-6}$; smaller error means a direction closer to the SVD reference. This quantity differs from the Pearson similarity used by admission and cannot be compared directly with the 0.90 threshold. The experiment also diagnoses relative eigen-residuals of $RR^\top s$ against the direction implied by its Rayleigh quotient; residuals require two extra products.

Among 9388 matrices, 13.9% have $\sigma_2/\sigma_1 > 0.9$. With seed 0, four-step power iteration misses the strict target on 7053 matrices. Figure 12 compares iteration budgets and actual product counts: Lanczos approaches the SVD reference faster as the budget grows. Controlled-spectrum experiments also show slower power convergence when the two leading directions are closer. This measures finite-budget approximation, not whether the implementation is faulty.

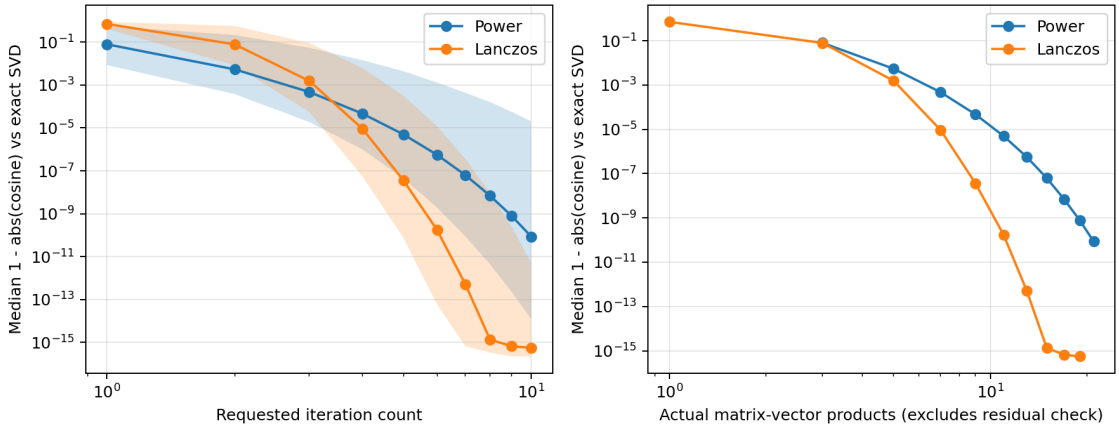


Figure 12: Direction errors of power iteration and Lanczos against iteration count (left) and actual matrix–vector products (right).

6.4.2 Cost

SVD first determines an offline iteration budget for each matrix, capped at ten steps. Both methods process all 9388 matrices; unreached targets are reported separately. Costs are then compared on the 6679 matrices where both methods reach the target (Table 13, Figure 13). This common reached set means both methods attain the same error threshold within ten steps, avoiding a timing comparison at different accuracies. Lanczos reaches more matrices and reduces common-set solve time from 10.235 s to 8.154 s, about 20.3%.

Table 13: Matched-accuracy comparison. Reached/unreached counts cover all matrices; the final three columns cover only the common 6679. Logical payload uses decimal GB.

Method	Reached	Unreached at 10	Solve / s	Allreduce calls	Payload / GB
Power	6679	2709	10.235	78747	1.694
Lanczos	8179	1209	8.154	109162	1.127

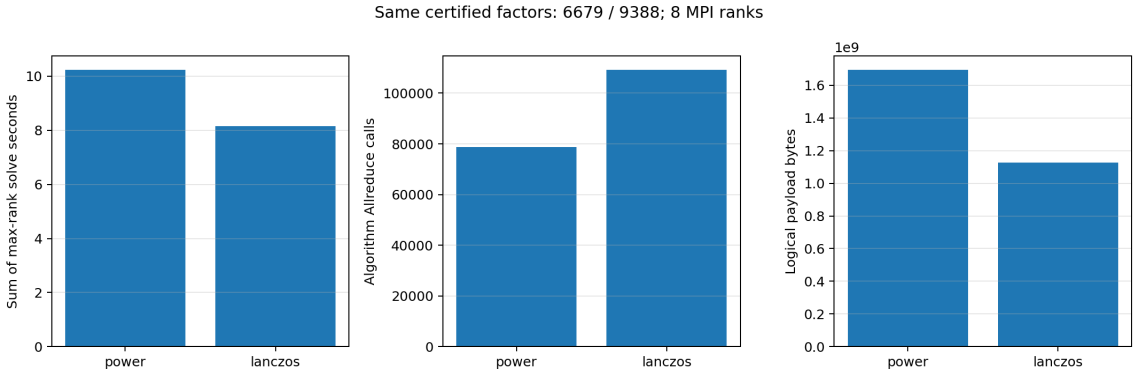


Figure 13: Solve time, reduction calls and logical communication payload for the common reached set.

Lanczos reorthogonalisation adds scalar reductions, but the required iteration budgets also differ; more reduction calls can still carry fewer total bytes. Logical payload is not network traffic: it sums reduction-array bytes, whereas physical transmission also depends on MPI's communication algorithm. The experiment reserves 16 cores and 111 GB but uses 8 single-threaded MPI ranks. Both methods reduce per factor, without the production power solver's batched reductions. Pure solve timing excludes residual diagnostics, input caching and logging. This compares offline-selected budgets; production still uses a fixed step count.

6.4.3 Effect on admission decisions

Use four-step power iteration with seed 0 as the baseline. Candidate order and quality are fixed; only the signature method and budget change. Action changes count differences between admit, replace and reject; nearest-record changes also include cases where the action stays the same but the matched record differs. Table 14 shows that eight-step power changes 637 actions and gives a final-membership symmetric difference of 372. This counts both members entering and leaving, not replacement events: one member removed and another added contribute 2. The complete experiment stores 60 trajectories, and different starts also give changes (Figure 14).

Table 14: Admission changes relative to the four-step power baseline.

Method	Steps k	Action changes	Nearest-key changes	Final symmetric diff.
Power	2	960	5984	568
Power	4	0	0	0
Power	8	637	4204	372
Lanczos	2	1373	6992	942
Lanczos	4	469	3702	233
Lanczos	8	758	4945	461

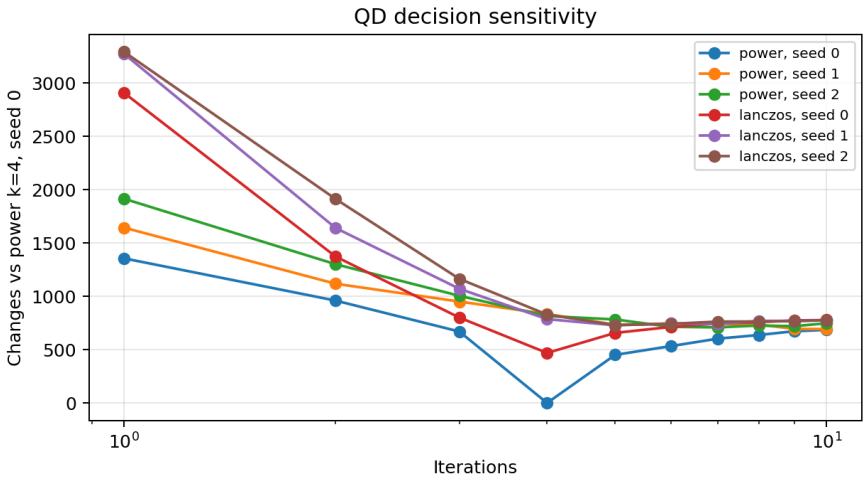


Figure 14: Effects of solver, budget and starting vector on admission actions.

Reproducing the original four-step baseline and preserving its membership after choosing a more accurate direction are different goals. A sequential archive propagates early decision differences; lower direction error alone does not establish better factor quality.

7 Discussion

7.1 How each part is parallelised

Table 15 summarises parallelism and communication. The engine partitions dates, keeping cross-sectional work local and exchanging history for rolling operators. PCA follows the date partition but also merges norms and right vectors each iteration. GPUs accelerate ranking, multiplication and similarity; archive updates retain sequential dependencies.

Table 15: Parallel methods, logical communication and bottlenecks. Halo is per rank; other volumes are per batch.

Module	Parallelism	MPI volume or operation	Bottleneck
Factor operators	MPI + OpenMP	halo: 1.5–1.7 GB per rank/batch; ema: $p-1$ sends of $2N$ values	History and imbalance
RankIC / fitness	Local dates, global summaries	Reduce $O(E)$ statistics	Ranking / bandwidth
Sample ranking	CPU / CUDA	No distributed ranking exchange	Sorting / transfers
PCA (date)	MPI + CPU/CUDA	$B + BN$ values per iteration, then a final B	Fixed reduction size
PCA (transpose)	Alltoallv + local solve	Redistribute $O(BmN)$, gather $O(Bm)$	Preparation / reading
Central archive	CPU/CUDA estimates, CPU refinement	Gather $O(Bm)$ signatures	Sequential admission
Sharded archive	MPI comparison + MAXLOC	One MAXLOC per candidate	Communication / metadata

Halo dominates logical sending in the original engine batch: about 500 GB in total at 288 ranks, while IC-summary reductions take little time. PCA has a different limit: the right-vector

reduction in the date layout does not shrink as ranks increase. Actual GP also includes startup, snapshot preparation, Python control and checkpoints, so resources should be chosen by the target workflow's walltime, not by a single kernel.

7.2 Conclusions about the data

The real-data universe grows from 2636 to 4845 active stocks, so equal date blocks have unequal work; this is an important source of engine waiting and scaling loss. Fixed-size R experiments measure PCA cost rather than that changing engine load. Distributed reductions and panel precision can also affect nearly equal values; numerical agreement, admission agreement and complete evolution outcomes are checked separately.

Table 16 and Figure 15 examine the random depth-3 proposals: 8866 valid candidates and the 2988 archive members obtained with four-step power. This is not the earlier three-depth GP's 325 members, nor the historical library of 1481 selected factors used in validation.

Table 16: In-sample absolute mean RankIC of random proposals and the QD archive.

Set	Count	Minimum	25%	Median	75%	Maximum
Valid candidates	8866	1.82136×10^{-7}	0.0086576	0.0230372	0.0434855	0.107231
QD archive	2988	1.82136×10^{-7}	0.00349045	0.0107784	0.0240051	0.107231

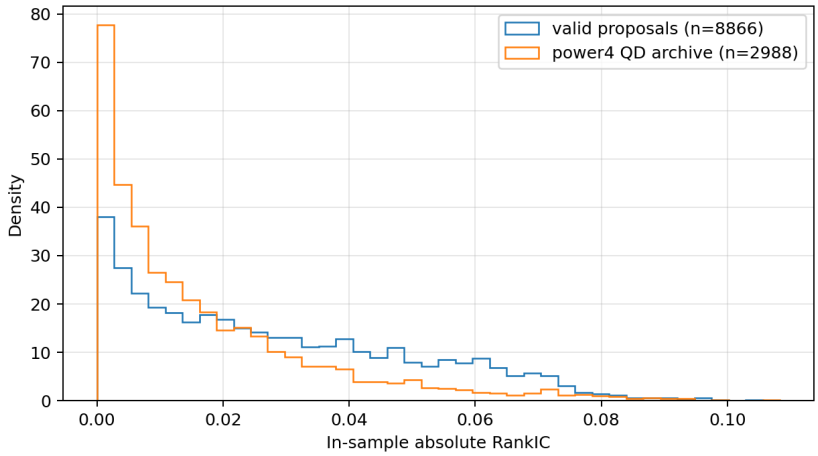


Figure 15: In-sample RankIC distributions of valid candidates and the QD archive.

The archive median is below the candidate median. QD compares quality within similar behaviour patterns rather than keeping only the highest-RankIC fraction; a lower-quality factor can enter if its pattern differs. The distribution illustrates a quality/diversity trade-off, not investment returns or out-of-sample performance.

Precision affects ranking too: distinct float64 values can become tied after float32 conversion. Panel-storage effects must therefore be separated from solver differences on the same FP32 R. RankIC is checked using identically rounded inputs; solver comparisons check signatures and complete admission trajectories. FP32/FP64 solving changes neither actions nor final members for this batch, without implying agreement for every possible input.

7.3 Future work

The factor engine can use work-weighted date blocks to reduce waiting as the stock universe grows, organise cross-block ema propagation with `MPI_Exscan`, and fuse element-wise operators to reduce intermediate panels and memory traffic.

GP can explore asynchronous steady-state evolution, overlap CPU factor evaluation with GPU PCA, and prepare random immigrants before the current generation's PCA finishes. Independent subprocess and archive-snapshot overhead can also be reduced. Chunking should follow the device-memory budget: with only 24 GB, smaller batches and on-demand uploads avoid retaining too many root panels at once. Residual-based online stopping may reduce unnecessary iterations, but diagnostic cost and admission changes must be measured together; lower direction error alone is not a complete workflow evaluation.

8 Conclusion

We move the original Python GP's batch evaluation into C++, share subexpressions through a DAG, partition dates for an MPI + OpenMP engine, and integrate handwritten CPU/CUDA PCA, similarity and sequential QD. The original fixed engine batch falls from about 5060 s on one core to 75.0 s with 288 pure-MPI ranks. On the same 288 cores, node-level threading with 4×72 reaches 27.8 s. Gains come from less local work and shared-memory reuse; halo, boundary restarts and imbalance limit later scaling.

PCA experiments show why ranking, preparation, communication and admission must be timed alongside multiplication. Two A40s are 1.764 times faster than one, but the 72-core CPU configuration is faster for this batch workflow. Lanczos reaches the target on more matrices within ten steps and reduces common-target solve time by about 20.3%; changing the solver or budget also changes some archive members. Three-depth GP runs at full data dimensions and the shared 16-core calibration further verify the connection between factor evaluation, PCA, QD, subsequent evolution and checkpoints.

Use of AI tools

ChatGPT was used to assist discussion.

A Code structure, building and running

The main sources are in the project's GitHub repository (`pca-implementation` branch). The project comprises the factor engine, DAG analysis tools, Python GP and PCA backend. Table 17 lists the main directories and entry points.

Source or directory	Role
`include/gpe/`	Public C++ interfaces for panels, DAGs, operators, evaluation and distributed computation
`src/core/`	Parsing, DAG merging, panel I/O, operators, RankIC and fitness, including OpenMP parallelism
`src/mpi/distributed.cpp`	Date partitioning, halo exchange, ema state propagation and MPI batch evaluation
`apps/gpe_bench.cpp`	Engine benchmark driver: panel/expression input, timing and scaling results
`dag/`	`dag_stats.py`, `dag_runtime.py`, `memory_sim.py`, `cache_sim.py` for DAG statistics, planning and memory/cache analysis
`python/src/gp_factor_miner/`	Original GP: data loading, expressions, hierarchical evolution, selection/mutation, archive, cache and checkpoints; CLI: `hierarchical_cli.py`
`python/scripts/`	Panel/reference export, Python benchmarks, factor analysis and plotting
`pca/core.cpp`, `pca/gpu.cu`, `pca/main.cpp`	CPU power/Lanczos, CUDA ranking/products, similarity, sequential admission and standalone PCA driver
`pca/engine_bridge.cpp`, `pca/gp_bridge.py`, `pca/run_gp.py`	Factor-engine/PCA integration and the three-depth GP driver for existing fields
`tests/`, `python/tests/`	C++ engine and original Python GP tests; DAG/PCA-specific tests are in `dag/` and `pca/`
`scripts/vanda/`, `pca/*.pbs`	Vanda environment, MPI launch, binding, build/validation and job scripts
`docs/`, `report/`	Module/user documentation, LaTeX reports and figures

Table 17: Main sources and entry points in the GitHub repository.

The root `CMakeLists.txt` builds the C++ factor engine; `pca/CMakeLists.txt` builds PCA separately. Python dependencies are declared in `python/pyproject.toml`. Run commands from the repository root, and build/run on compute nodes when using the cluster. The original engine's Vanda commands are retained at the end of this appendix; build PCA and its live GP backend with:

```
python3 -m pip install -e './python'
cmake -S pca -B pca/build_live -DPCA_MPI=ON -DPCA_ENGINE_BENCH=ON
cmake --build pca/build_live -j 4 --target pca_engine_bridge pca_run
```

Use a separate build directory with `-DPCA_CUDA=ON` for CUDA, matching MPI ranks to the allocated GPUs. PCA production targets use handwritten code; `PCA_CUBLAS_REFERENCE` adds only the separate cuBLAS performance reference.

Choose the entry point by workload: `gpe_bench` for a fixed expression batch, the scripts in `dag/` for DAG analysis and planning, `python3 -m gp_factor_miner.hierarchical_cli` for original EOD data, or `pca/run_gp.py` for existing evolution-period binary fields. After building the root CMake project, an engine batch benchmark has the form:

```
OMP_NUM_THREADS=1 mpirun -np 4 build_vanda/gpe_bench \
  /path/to/panels /path/to/expressions.json --threads node --chunk 1000
```

The following existing-field command reproduces this report's population-64 GP with one generation at each of three depths:

```
python3 pca/run_gp.py --panel-dir /path/to/evolution/panels \
  --engine-binary pca/build_live/pca_engine_bridge \
  --pca-binary pca/build_live/pca_run \
  --out pca/runs/example_gp_01 --ranks 16 --threads 1 \
  --population 64 --generations 1 --gene-pool 32 \
  --warm-start-multiplier 3 --progress-chunk 1000 \
  --windows 1 5 10 15 20 30 60
```

See `docs/dag/README.md`, `docs/engine/data_format.md` and `pca/GP_INTEGRATION.md` for DAG tools, input format and GP operation. The original engine's Vanda build, validation, experiment-submission and plotting commands follow:

```
module load OpenMPI/5.0.8-GCC-14.3.0 CMake/4.0.3-GCCcore-14.3.0
cmake -S . -B build_vanda -DGPE_ENABLE_MPI=ON
cmake --build build_vanda -j
qsub scripts/vanda/build_test.pbs # build and validation tests
bash scripts/vanda/submit.sh # experiment jobs
python python/scripts/plot_vanda.py <csv_dir> <figure_dir>
```